

LLM-Assisted Security Assessment of Open-Source Software Repositories

Comprehensive Project Report

Alireza Shahidiani | MSc in Artificial Intelligence, University of Bologna

Curricular internship project | 15 September 2026

Executive summary

This project develops a Python application for assessing the security of open-source repositories using large language models (LLMs), conventional analysis tools and explicit evidence reporting. It covers code vulnerabilities, malicious behaviour, exposed secrets and vulnerable dependencies. The application collects repository content, preprocesses files and functions, performs bounded analysis, and generates reports that distinguish findings from incomplete or failed checks.

The experimental programme combines a multi-language vulnerability corpus, category-specific benchmarks and repository-level evaluation. The corpus contains 35,952 functions associated with 3,472 CVEs. On a 582-record vulnerability set with unambiguous sample identities, the tested model/prompt configurations achieve F1 scores of 0.273–0.307. In the repository benchmark, target files are selected in 17 of 56 verified cases and receive vulnerability alarms in 14; seven cases have a finding attached to the target function body. These are coverage and location measures, not independently adjudicated correct-defect counts.

On synthetic malware, Qwen produces fewer benign false alarms than the installed YARA subset, while YARA detects slightly more malicious samples. Secrets and dependency results show different trade-offs, illustrating why security categories should be assessed separately. A controlled Python experiment measures the response to code reserialisation and literal splitting. The report presents this experiment, its control and its scope explicitly.

The project delivers a working analysis workflow and reproducible research artifacts. Its intended role is to support a reviewer, not certify that a repository is safe. Outstanding scientific questions concern independent labels, unseen threat families and real-world reviewer benefit. They do not negate the implementation, but they set boundaries on the claims supported by its results.

1. Project objectives and scope

The internship investigates whether LLMs can contribute to automated security assessment of public software repositories. The objectives are to build a Python analysis pipeline, examine multiple security categories, compare observed results with conventional tools, study false alarms and explanatory output, and report the findings with appropriate limitations.

The project focuses on source-level evidence. It does not execute malware fixtures, validate live credentials, prove exploitability or certify deployment safety. A vulnerability in application code and a malicious installation action are different risks: the former may affect users of the software, while the latter may threaten the person installing or analysing it. The application distinguishes these contexts rather than treating every warning as evidence that a repository is malicious.

The central questions are: what does the analyser inspect; what warnings does it produce; how do those warnings compare with labelled examples and tool outputs; and how reliable are the measurements? High model scores are not an objective in themselves. An informative negative result is a valid outcome when the experiment and its interpretation are sound.

2. Background and research basis

LLMs provide a shared interface for code-oriented tasks but their fluent output is not proof of security correctness. Sheng et al. [1] describe variation in vulnerability-detection datasets and evaluation methods. Steenhoek et al. [2] examine difficulties reasoning about security-relevant semantic differences. SecVulEval [3] emphasises contextual information, benchmark quality and fine-grained localisation. These works motivate the project's emphasis on evidence, context and transparent denominators.

The project's contributions are an integrated analysis application, a consistent multi-language collection pipeline, an empirical comparison across four categories, and evaluation controls for sample identity, selected-code coverage and paired inputs. It does not claim a novel foundation model or universal superiority over existing scanners. The malware baseline uses installed YARA rules associated with GuardDog, whose full package-analysis functionality is broader than snippet-level rule matching [4].

3. Application architecture

3.1 Processing pipeline

The application accepts a public repository address or a local directory. Repository collection obtains source content; preprocessing identifies supported files, excludes unsuitable inputs and extracts functions where applicable. Static-guided ordering prioritises files for bounded AI analysis. Category-specific modules then produce findings and execution status. Aggregation combines those findings into a report with locations, classifications, severity, explanations and coverage information.

The research repository benchmark fixes the AI file budget at 25. Vulnerability analysis also limits inspection to 12 extracted functions per file. Consequently, selecting a file is not equivalent to examining every function in it. Oversized, unreadable, unsupported and budget-excluded content must remain visible in coverage accounting.

3.2 Four security categories

Vulnerability analysis asks whether a function contains a security weakness and requests CWE classification. The supplied function context limits what can be inferred about callers, sanitisation and cross-file behaviour. Malware analysis asks whether capabilities such as networking or process execution indicate harmful intent in context. Legitimate use of those capabilities is not automatically malicious.

Secrets analysis evaluates candidate sensitive strings and their context; synthetic evaluation distinguishes labelled secrets from documentation examples, placeholders and other negatives. Dependency analysis combines manifest extraction with advisory information. A database match requires applicability checks, and a model-generated package name that does not occur in a manifest is recorded separately rather than ignored in review-burden accounting.

3.3 Models, conventional tools and comparison

The framework supports model-provider selection and local inference, including Qwen through Ollama. The evaluated vulnerability configurations use Qwen and a Google model recorded under the project alias gemini-flash. Tool integration includes Semgrep, Bandit and Gitleaks where installed, as well as category-specific rule/database checks. Availability, errors and unsupported inputs affect coverage and are not negative security findings.

The comparison workflow associates findings with compatible classifications and locations. Agreement means two systems report compatible observations, not that the observation is correct. An unmatched warning in code that another system did not inspect cannot be treated as that system's false negative. No matched conventional-tool vulnerability accuracy benchmark is claimed from interface-level overlap alone.

3.4 Evidence and report semantics

The report separates model opinions, corroborated observations and deterministic tool or database output. This hierarchy supports review decisions but does not guarantee correctness: deterministic tools can also report false positives. A failed or incomplete scan must not generate an unconditional clean conclusion. The relevant states include completed, partial, failed and unknown coverage.

Findings retain file locations, categories and supporting explanations. Sensitive credential values are redacted. The summary should be read together with the scope and unresolved checks. Natural-language explanations are an aid to investigation; they do not constitute proof of exploitation or faithful access to a model's internal reasoning.

4. User workflow and operational behaviour

Start the application with the project Python environment and select a repository or local folder. Choose supply-chain checks, AI analysis, conventional static tools, or the comparison workflow. Select a real model for research-quality inference; the stub produces demonstration responses and is not a security baseline.

The file budget bounds AI workload. Validation-sample calibration can select a vulnerability strategy for interactive scans, but a small calibration set is only a rough ranking and is not a performance estimate for the scanned repository. Other categories use their own prompts. The experiments reported here use fixed configurations so their results are not described as validation-selected winners.

During processing, progress and execution status are recorded. Cancellation occurs at an analysis boundary and may wait for a model request or tool invocation. Saving creates a report artifact; cancelling a save does not imply that no temporary output exists. Reports explain coverage before findings so users can distinguish no reported warning from no successful inspection.

For a practical review, first inspect status and coverage, then examine high-consequence findings in their source context, and finally verify unresolved questions with appropriate tools or human expertise. This is the intended workflow; a formal study of reviewer time savings or decision quality has not yet been conducted.

5. Dataset construction and quality controls

5.1 Vulnerability corpus

The collection pipeline uses public vulnerability metadata and fixing commits. Reverse application of a patch reconstructs pre-fix source; function extraction produces records with language, repository, commit, CVE, CWE, function text and identity fields. Reconstruction checks ensure the recorded body matches the recovered text. They do not independently establish that the function is the actual location of the security defect.

The corpus contains 35,952 functions: 5,136 labelled vulnerable and 30,816 labelled safe, from 1,489 repositories, with 3,472 CVEs and 196 CWE categories. C, C++, Java, Python and JavaScript are represented. Post-fix and unchanged functions are assumed negatives under the construction procedure. The observed class balance is a property of this sampling procedure, not an estimate of security-defect prevalence in public software.

Class distribution (N = 35,952)

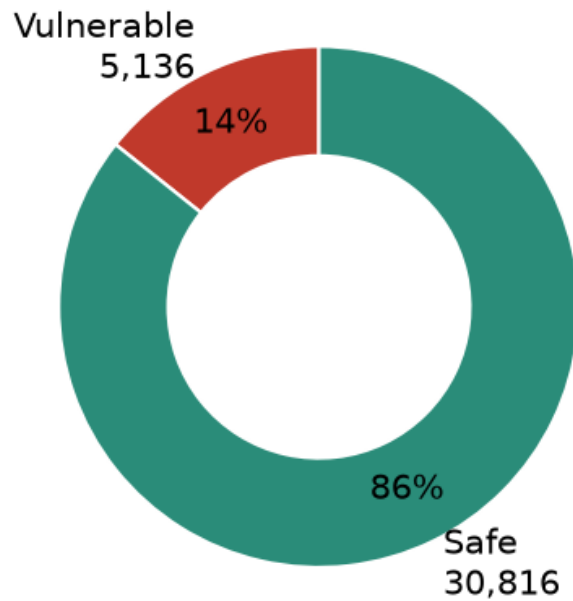


Figure 1. Label composition of the constructed corpus. Labels reflect the extraction procedure and require contextual validation.

Sample distribution by programming language

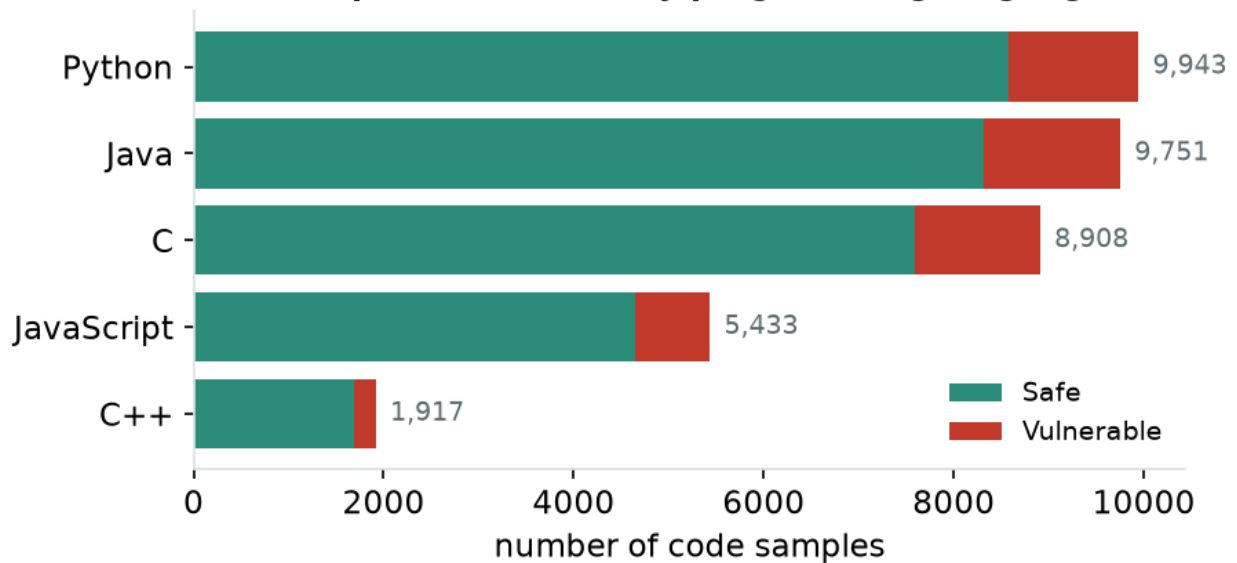


Figure 2. Language composition of the corpus, separated by recorded label. This is corpus composition, not per-language detector performance.

Training, validation and test partitions contain 26,108, 3,542 and 6,302 records. Repository, pair and normalised exact-hash overlap checks pass. Four CVEs occur in both training and test and one in training and validation. A heuristic token-shingle search flags 12 candidate near-duplicates among 5,741 eligible test records. These candidates are not yet human-adjudicated; neither exact-hash disjointness nor this limited screen proves semantic independence.

5.2 Identity and label validity

The evaluated vulnerability sample has 600 records per configuration. Legacy identifiers collide for repeated or overloaded function names; corpus linkage is ambiguous for 18 rows. The reported sensitivity set therefore uses 582 unambiguous records across 167 repositories, comprising 106 positive and 476 negative labels. Ambiguous rows are excluded rather than assigned guessed identities. Future records use content-backed identities and source hashes.

A human review of 50 diversity-selected positive cards yields 20 correct labels, 13 wrong-CWE judgements, 12 not-vulnerable judgements and five unclear cases. The selection is round-robin across categories, not population-random, and excludes negative labels and orphaned positives. These results identify a label-validity problem but cannot estimate its corpus-wide prevalence.

The secondary LLM review has 68% exact four-category agreement with the human pass and kappa 0.521. Three-way agreement is 86% when correct and wrong-CWE are grouped as vulnerable. Binary agreement is 42/42 only after excluding eight cards with an unclear verdict from either reviewer. Review timing and blinding are unrecorded, so independent human validation is not claimed.

5.3 Auxiliary benchmarks

The malware benchmark contains 400 synthetic examples, with 300 included in the category comparison. The 339 eligible Python records supply the paired experiment; 61 non-Python records are outside its scope. Synthetic variants within a family are correlated, and there is no established one-to-one malicious/benign twin design.

The secrets benchmark contains 300 files with labelled candidate items. The dependency benchmark contains 300 manifests and 4,243 listed entries. Their units differ from malware classification, so scores should be compared within a category. The dependency oracle shares advisory-derived information with the benchmark labels and is an internal-consistency reference, not independent evidence of perfect real-world detection.

6. Experimental methods

The vulnerability experiment compares zero-shot and eight-step chain-of-thought prompts for Qwen and the recorded Google configuration. The 600 selected identities match sampling from the saved test set with seed 42. Exact historical cloud snapshots and invocation records are incomplete; current source snapshots cannot automatically recover those missing details.

Precision, recall, F1, accuracy, balanced accuracy and Matthews correlation coefficient (MCC) describe prediction behaviour. An always-negative baseline exposes the effect of class imbalance. Vulnerability intervals use 2,000 repository-cluster bootstrap resamples. Exact paired correctness tests are adjusted across six configuration comparisons using Holm correction. A supplementary 10,000-swap repository-block permutation analysis uses seed 42. These analyses are post-hoc and assume appropriate repository-block independence and exchangeability; they do not test F1 superiority or universal model equivalence.

The repository benchmark attempts 60 cases reconstructed at recorded commits. All target functions must verify before a case is scored. The selected-file trace comes from the deployed scan. A target-file alarm hit means at least one vulnerability warning appears anywhere in a target file. Body-location and CWE-match measures add specificity but still do not prove the warning identifies the benchmark defect. Unreviewed off-target alarms are review burden, not known false positives.

The paired Python experiment checks source hashes, labels and parsed ASTs. Its transformation reserialises the AST and splits ordinary string literals. Formatting and comments can change; docstrings, formatted strings and match constructs are preserved by the transformer. A reserialisation-only control separates this background change from the additional literal transformation. Both detectors evaluate the same eligible identities; missing responses are reported as failures, not benign classifications. No input is executed.

7. Results and interpretation

7.1 Vulnerability prediction

Table 1. Vulnerability sensitivity set: 582 records per configuration, 167 repositories. F1 intervals use repository-cluster bootstrap resampling.

Model / prompt	Accuracy	Balanced acc.	F1	MCC	95% F1 interval
Qwen / zero-shot	0.639	0.559	0.305	0.097	0.243–0.376
Qwen / CoT	0.582	0.550	0.304	0.079	0.233–0.375
Gemini / zero-shot	0.605	0.557	0.307	0.090	0.226–0.390
Gemini / CoT	0.634	0.534	0.273	0.057	0.203–0.347

All configurations show modest discrimination. The always-negative baseline reaches 81.8% accuracy while finding no vulnerabilities, demonstrating why accuracy alone is misleading. Qwen zero-shot has 46 true positives and 150 false positives; chain-of-thought has 53 and 190 respectively. The accuracy difference favours zero-shot by 5.67 percentage points, while recall moves in the other direction.

Only this Qwen contrast survives the six-comparison adjustment: paired adjusted $p=0.00268$ and repository-block adjusted $p=0.02820$. This is evidence about the selected configurations and endpoint, not a general conclusion that chain-of-thought damages reasoning. The other comparisons do not establish either a reliable difference or equivalence. Independent labels and richer context may affect results, but their causal contribution has not been isolated.

7.2 Malware, secrets and dependencies

Table 2. Historical auxiliary results. Best-observed selection is exploratory; categories have different evaluation units. Dependency scores here cover listed entries.

Category	Configuration	Precision	Recall	F1
Malware	guarddog_yara	0.706	0.852	0.772
Malware	Qwen / zero_shot	0.992	0.819	0.898
Secrets	gitleaks	1.000	0.802	0.890
Secrets	Qwen / zero_shot	0.497	0.877	0.635
Dependencies	osv_lookup	1.000	1.000	1.000
Dependencies	Gemini / zero_shot	0.281	0.197	0.232

The malware comparison favours Qwen on F1 and false alarms, while the YARA subset has higher recall. For secrets, Qwen has higher recall but substantially lower precision and F1 than Gitleaks. These observations do not support a single best analyser across all categories. The best-observed LLM configurations are selected descriptively from evaluated results; no confirmatory superiority test is claimed.

Dependency extraction is particularly weak relative to the shared-source oracle. An additional burden is invented package names: Qwen chain-of-thought reports 329 packages not listed in the manifests, versus 14 for zero-shot. Counting those names as additional false positives reduces chain-of-thought listed-entry F1 from 0.108 to an all-reported F1 of 0.087. The all-reported scoring option exposes this error category rather than silently omitting it.

7.3 Repository-level coverage and alarm hits

Table 3. Instrumented repository results: 60 attempted cases, 56 verified, 48 repositories. Rates are descriptive.

Measure	Count	Rate
Target-file selection	17/56	30.4%
Target-file alarm hit	14/56	25.0%
Alarm hit given selection	14/17	82.4%
Exact-body localisation	7/56	12.5%
Strict CWE agreement	3/56	5.4%

Four cases fail reconstruction; the 56 scored cases span 48 repositories. The recorded 82.4% alarm-hit rate conditional on selection describes a selected subset, not an experiment proving that a different ranking would recover the remaining defects. The 25-file budget and within-file function cap constrain coverage. Body-location matches occur in seven cases and strict CWE matches in three.

The mean 31.68 off-target alarms per case have not been independently adjudicated. The stored benchmark results do not retain individual warning explanations, which prevents retrospectively verifying correct-defect counts from those result files alone. Future benchmark outputs now retain the findings needed for that review. This evidence-retention repair does not manufacture missing historical evidence.

7.4 Python transformation and formatting control

Table 4. Paired Python experiment: 178 malicious and 161 benign items, no failed pairs for either detector.

Detector	Input	Malicious recall	Benign false alarms	F1
Qwen	original	143/178 (80.3%)	1/161 (0.6%)	0.888
Qwen	transformed	144/178 (80.9%)	0/161 (0.0%)	0.894
YARA subset	original	148/178 (83.1%)	76/161 (47.2%)	0.736
YARA subset	transformed	148/178 (83.1%)	76/161 (47.2%)	0.736

The 339 pairs comprise 178 malicious and 161 benign samples. All detector pairs complete successfully. Of these, 312 have an AST change and 27 are formatting-only cases. Qwen changes two decisions under the combined transformation: one more malicious item is flagged and one benign alarm disappears. YARA changes none. On the AST-changed subset, Qwen false positives move from 1/134 to 0/134 and YARA remains at 63/134.

Table 5. Three-arm comparison on matched identities and complete responses. The formatting control uses the same source, fixture eligibility, recorded model settings and rule files.

Detector	Input	Malicious recall	Benign false alarms	F1
Qwen	Original	143/178 (80.3%)	1/161 (0.6%)	0.888
Qwen	Reserialised only	144/178 (80.9%)	0/161 (0.0%)	0.894
Qwen	Reserialised + split	144/178 (80.9%)	0/161 (0.0%)	0.894
YARA subset	Original	148/178 (83.1%)	76/161 (47.2%)	0.736
YARA subset	Reserialised only	148/178 (83.1%)	76/161 (47.2%)	0.736
YARA subset	Reserialised + split	148/178 (83.1%)	76/161 (47.2%)	0.736

The formatting-control run is `data/results/formatting_control/20260915-project-report`. For local-qwen-coder, 339 items have complete three-arm evidence; repeated original predictions disagree on 0 items, and the reserialised-only and split inputs differ on 0 verdicts. For yara, 339 items have complete three-arm evidence; repeated original predictions disagree on 0 items, and the reserialised-only and split inputs differ on 0 verdicts. This is a sequential, cache-enabled control on the same synthetic sample, not an independent replication or a test of unseen threat families. The control reused 366 of 678 Qwen responses.

The two Qwen decision changes are already present under reserialisation alone. Adding literal splitting produces no additional verdict changes for either detector on these inputs. The observed improvement therefore cannot be attributed specifically to literal splitting. This control resolves that attribution ambiguity for the saved experiment, while leaving performance on other transformations and unseen examples untested.

The installed YARA baseline compiles 51 of 54 rule files under YARA 4.5.4. Three rule compilation failures are documented; the baseline is therefore a defined rule subset, not the full GuardDog scanner. Qwen uses the 7B model, temperature zero and an 800-token output budget. Cache reuse is recorded and does not provide independent repeat-run observations. The experiments address simple static transformations of synthetic Python, not new threat families or general adversarial robustness.

7.5 Explanations and operational reliability

Across 451 Qwen findings, weighted identifier grounding is 0.521; 1.8% of explanations mention at least one identifier absent from the analysed function. These measures describe textual grounding, not explanation correctness or causal faithfulness. A fresh human explanation study is needed to evaluate factual quality and usefulness.

On 35 repositories used during development, the system produces no clone-unsafe verdicts, while 11 have lower-severity alarms. These results are useful regression observations but are not held-out false-positive performance. Popularity or a trusted designation does not prove a repository is clean.

Software verification covers failure handling, coverage, typed parsing, finding comparison, identity checks, statistics and paired evaluation. Fifty-eight regression tests pass in the current validation run. Their success establishes specific engineering properties; it does not demonstrate detection accuracy, complete application reliability or user benefit.

8. Limitations, resolutions and priorities

Measurement defects must be addressed before relying on their scores. The project now uses content-backed identities, actual selection traces, strict paired-input validation, explicit failure accounting and recorded baseline compilation status. The additional formatting control addresses the interpretation of the combined transformation. Future repository outputs preserve individual findings for review. These are concrete controls, not promises that all data are correct.

Some limitations require people or new evidence. The 50-card review cannot validate the whole corpus; negative labels and uncertain cards still need independent adjudication. The 14 target-file alarm hits cannot be called 14 correct defect detections without reviewing the underlying findings. Fresh repository cases are needed for false-alarm estimates, and human reviewers are needed for usefulness and explanation assessment. Broader model coverage and new threat families would strengthen generalisation but are not prerequisites for reporting the present experiments accurately.

Pretraining exposure cannot be inferred from repository-disjoint splits. Newly authored test families can be held out from this project's development but cannot establish every closed model's training history. The report therefore makes no zero-day-detection claim. Likewise, statically matched ASTs do not prove all observable runtime behaviour, particularly source introspection.

The next scientific priorities are a matched conventional vulnerability baseline, independent review of labels and alarm cases, and a prospectively held-out threat-family test. Those should follow a frozen protocol with

declared budgets. Repeating every experiment or adding more interface features would not by itself resolve these limitations. A tutor can assess whether the completed investigation and documented boundaries meet the internship's educational scope.

9. Reproducibility and deliverables

The delivered project includes the desktop application, data-collection and preprocessing modules, category evaluators, repository benchmark, paired-control evaluator, result files, tests, research notes and report builders. Experiments write separate outputs with manifests and source/input hashes. Existing results are preserved rather than overwritten by a new run.

The repository result set is `data/results/repo_bench_v2/20260915-022618`. The combined-transformation result set is `data/results/perturbation_v2/20260915-025616`. The formatting-control result set is explicitly identified in the control table. Model-response cache policy, failed checks, compiled rules and relevant identity assertions are part of the evidence. The report build pins these sources and does not silently select the latest folder.

To reproduce this document, run `python research/report/build_project_report.py` with the project Python environment and saved evidence. Rebuilding a PDF does not launch new inference. Historical model snapshots that were not recorded remain unknown. No public artifact archive or external artifact evaluation is claimed. Authorship, any redistribution of source examples and submission details require author/tutor review before public release.

The internship deliverables include implementation, experimental analysis, literature study and scientific communication. Completion of hours, attendance validation, questionnaires and tutor approval must be confirmed separately from the technical work. A research limitation is not automatically an unmet administrative requirement, nor does a working program establish completion of all internship obligations.

10. Conclusion

The project provides an integrated, evidence-aware workflow for LLM-assisted security analysis of open-source repositories. It supports multiple security categories and produces inspectable reports with explicit coverage and execution status. Experimental results show task-dependent strengths and weaknesses: modest function-level vulnerability prediction, limited repository target coverage, fewer synthetic-malware false alarms for Qwen than the evaluated YARA subset, and weaker LLM results for secrets and dependencies under the reported comparisons.

The principal outcome is a working research system with measured capabilities and clear boundaries. Correct identities, valid transformations, retained findings and appropriate statistical units make the results more interpretable; they do not turn uncertain labels or unreviewed warnings into proven vulnerabilities. The project supports further investigation of reviewer-assisted security workflows without claiming autonomous safety or universal detection performance.

References

- [1] Ze Sheng et al. 2025. LLMs in Software Security: A Survey of Vulnerability Detection Techniques and Insights. <https://arxiv.org/abs/2502.07049v2>
- [2] Benjamin Steenhoek et al. 2025. To Err is Machine: Vulnerability Detection Challenges LLM Reasoning. <https://arxiv.org/abs/2403.17218v2>
- [3] Md Basim Uddin Ahmed et al. 2025. SecVulEval: Benchmarking LLMs for Real-World C/C++ Vulnerability Detection. <https://arxiv.org/abs/2505.19828>
- [4] Datadog. GuardDog project documentation. <https://github.com/DataDog/guarddog>